

Projeto Multidisciplinar - Back-End

Raizes do Nordeste API

1. Introducao

A rede Raizes do Nordeste precisa de uma API para centralizar pedidos feitos em diferentes canais: aplicativo, totem, balcao, pickup e web. A solucao entregue organiza o fluxo de autenticacao, consulta de cardapio, controle de estoque por unidade, criacao de pedidos, pagamento mock e atualizacao de status operacional.

O objetivo tecnico foi manter uma API executavel, documentada e facil de testar localmente, usando Rails em modo API, SQLite, migrations, seeds e contratos JSON.

2. Requisitos funcionais

Codigo	Requisito	Situacao
---	---	---
RF01	Cadastro de usuario cliente	Implementado em `POST /usuarios`
RF02	Login com token	Implementado em `POST /auth/login`
RF03	Perfis de acesso	Implementado com roles `cliente`, `atendente`, `cozinha`, `gerente` e `admin`
RF04	Consulta de unidades	Implementado em `GET /unidades`
RF05	Cardapio por unidade	Implementado em `GET /produtos?unitid=`
RF06	Criacao de pedido com itens e total	Implementado em `POST /pedidos`
RF07	Registro do canal do pedido	Implementado com `canalPedido` obrigatorio
RF08	Filtro de pedidos por canal	Implementado em `GET /pedidos?canalPedido=`
RF09	Atualizacao de status do pedido	Implementado em `PATCH /pedidos/:id/status`
RF10	Cancelamento de pedido	Implementado em `POST /pedidos/:id/cancelar`
RF11	Controle de estoque por unidade	Implementado em `GET /estoques` e `POST /estoques/movimentacoes`
RF12	Bloqueio por estoque insuficiente	Implementado na criacao do pedido e na saida de estoque
RF13	Pagamento externo mock	Implementado em `POST /pedidos/:id/pagamentos`
RF14	Fidelidade	Previsto como regra de negocio documentada; pode ser acoplado ao pagamento aprovado
RF15	Promocoes	Previsto como regra de negocio documentada; pode ser aplicado antes do total do pedido

3. Requisitos nao funcionais

Codigo	Requisito	Atendimento
---	---	---
RNF01	Persistencia	SQLite com Active Record e migrations
RNF02	Senha segura	`has_secure_password` e `password_digest`
RNF03	Autenticacao	Token assinado pelo Rails, com expiracao de 24 horas
RNF04	Autorizacao	Regras por perfil nos endpoints restritos
RNF05	Auditoria	Registros em `audit_logs` para login, pedidos, pagamentos, estoque e status
RNF06	Padrao de erro	Resposta padronizada com `error.code` e `error.message`
RNF07	Documentacao	README, OpenAPI, referencia de endpoints e colecao Postman
RNF08	Testabilidade	Testes de integracao Rails e colecao Postman
RNF09	Paginacao	Listagens com `page`, `limit` e `meta.total`
RNF10	Tolerancia a falhas	Pagamento mock registra retorno aprovado ou negado sem depender de provedor real

4. Priorizacao

O fluxo principal priorizado foi pedido -> pagamento mock -> status. Ele cobre a parte mais importante da operacao da rede: cliente autenticado, escolha de unidade/produto, validacao de estoque, registro do canal, baixa de estoque, pagamento e acompanhamento do pedido.

Fidelidade e promocoes ficaram documentadas como regras previstas, pois nao sao necessarias para fechar o fluxo critico pedido/pagamento/estoque. A estrutura atual permite incluir esses modulos sem alterar o contrato principal do pedido.

5. Arquitetura

O projeto segue MVC com servicos de aplicacao:

Camada	Arquivos principais	Responsabilidade
---	---	---
API	`app/controllers/api/v1`	Rotas, autenticacao, autorizacao, request e response
Dominio	`app/models`	Entidades, relacionamentos e validacoes
Aplicacao	`app/services`	Fluxos de pedido, estoque e pagamento

| Infraestrutura | `db`, `config`, `storage` | Banco, migrations, ambiente e persistencia |

Os servicos `Orders::Creator`, `Stocks::Mover` e `Payments::MockProcessor` concentram as regras que nao devem ficar espalhadas pelos controllers.

6. Diagramas

Artefato	Arquivo
Casos de uso	`docs/diagrama\_casos\_uso.svg`
Arquitetura	`docs/diagrama\_arquitetura.svg`
DER	`docs/der.svg` e `docs/der.md`
Classes de dominio	`docs/diagrama\_classes.svg`
Fluxo pedido/pagamento	`docs/diagrama\_fluxo\_pedido.svg`

7. Casos de uso e fluxo critico

Atores

- Cliente: consulta cardapio, cria pedido, paga e acompanha status.
- Atendente: acompanha pedidos de balcao e pode atuar no fluxo operacional.
- Cozinha: atualiza preparo e andamento do pedido.
- Gerente/Admin: gerencia unidades, produtos, estoque, usuarios e auditoria.
- Gateway de pagamento: representa a integracao externa simulada pelo mock.

Feature: realizar pedido e solicitar pagamento

Pre-condicoes:

- usuario autenticado;
- unidade ativa cadastrada;
- produto ativo com estoque na unidade;
- `canalPedido` informado.

Fluxo principal:

1. Cliente faz login e recebe token.
2. Cliente consulta unidades e produtos disponiveis.
3. Cliente cria pedido com `canalPedido`.
4. A API valida unidade, produtos, quantidade e estoque.
5. O estoque e baixado na criacao do pedido.
6. O pagamento mock recebe `APROVADO` ou `NEGADO`.
7. Pedido aprovado muda para `pago`.
8. Pedido negado muda para `pagamento\_negado`.
9. Admin, gerente, atendente ou cozinha atualiza status operacional.

Pos-condicoes:

- pedido fica registrado com cliente, unidade, canal, itens e total;
- estoque da unidade e atualizado;
- pagamento fica registrado com payload enviado e resposta do mock;
- acoes sensiveis ficam em auditoria.

Excecoes:

- token ausente ou invalido: `401 nao\_autenticado`;
- perfil sem permissao: `403 sem\_permissao`;
- canal ausente ou invalido: `422 pedido\_invalido`;
- produto indisponivel ou estoque insuficiente: `422 pedido\_invalido`;
- pagamento em pedido fora de `aguardando\_pagamento`: `422 status\_invalido`;
- cancelamento fora dos status permitidos: `422 pedido\_nao\_cancelavel`.

Idempotencia:

O fluxo atual nao cria chave idempotente. Em producao, a recomendacao e receber um identificador unico do

canal de origem para evitar pedido duplicado em tentativas de App, Totem ou Web.

8. Regras de negocio

- `canalPedido` e obrigatorio e aceita `APP`, `TOTEM`, `BALCAO`, `PICKUP` ou `WEB`.
- Cliente visualiza apenas os proprios pedidos.
- Admin e gerente podem criar unidade, produto e movimentar estoque.
- Listagens aceitam `page` e `limit`, com limite maximo de 100 itens por pagina.
- Pedido so pode ser criado com produto ativo e estoque suficiente.
- Pagamento so pode ser feito quando o pedido esta em `aguardando\_pagamento`.
- Cancelamento e permitido para `aguardando\_pagamento`, `pago` e `preparando`.
- Pagamento mock com `cardToken` igual a `NEGADO` registra pagamento negado.
- Fidelidade prevista: pontuar pedido pago e permitir resgate em pedido futuro mediante consentimento.
- Promocao prevista: desconto por campanha ativa antes do fechamento do total, com registro da regra aplicada.

9. LGPD e seguranc

Dados pessoais usados: nome, e-mail, senha criptografada e aceite de privacidade. A finalidade e autenticar o usuario, vincular pedidos ao cliente e permitir rastreabilidade operacional.

Controles aplicados:

- senha armazenada como hash;
- token obrigatorio em endpoints sensiveis;
- autorizacao por perfil;
- retorno de usuario sem `password\_digest`;
- `consent\_at` para aceite de privacidade;
- auditoria de acoes sensiveis.

Para uma evolucao em producao, a retencao de dados pode anonimizar clientes inativos apos prazo definido pela politica da rede, mantendo apenas dados fiscais ou operacionais indispensaveis.

10. API e testes

A referencia detalhada dos endpoints esta em `docs/api.md`. O contrato OpenAPI fica em:

GET /api/v1/openapi.json

Principais endpoints implementados:

Recurso	Metodo e rota	Permissao	Retorno esperado
Auth	POST /auth/login	publico	200 com token
Auth	GET /auth/me	autenticado	200 com usuario
Usuarios	POST /usuarios	publico	201 com cliente
Usuarios	GET /usuarios	gerente/admin	200 com lista
Usuarios	GET /usuarios/:id	proprio usuario/gerente/admin	200 com usuario
Unidades	GET /unidades	publico	200 com lista
Unidades	GET /unidades/:id	publico	200 com unidade
Unidades	POST /unidades	gerente/admin	201 com unidade
Produtos	GET /produtos?unitId=	publico	200 com cardapio
Produtos	GET /produtos/:id	publico	200 com produto
Produtos	POST /produtos	gerente/admin	201 com produto
Produtos	PATCH /produtos/:id	gerente/admin	200 com produto
Estoques	GET /estoques	autenticado	200 com saldos
Estoques	POST /estoques/movimentacoes	gerente/admin	200 com saldo atualizado
Pedidos	GET /pedidos?canalPedido=	autenticado	200 com pedidos
Pedidos	GET /pedidos/:id	dono/perfil interno	200 com pedido
Pedidos	POST /pedidos	autenticado	201 com pedido
Pedidos	PATCH /pedidos/:id/status	atendente/cozinha/gerente/admin	200 com status
Pedidos	POST /pedidos/:id/cancelar	dono/perfil interno	200 com cancelamento
Pagamentos	POST /pedidos/:id/pagamentos	dono/atendente/gerente/admin	201 com pagamento

Padrao de erro:

```
{  
  "error": {
```

```
"code": "codigo_do_erro",
"message": "Mensagem do erro."
}
}
```

A colecao Postman esta em:

docs/raizes_api.postman_collection.json

Ela cobre login, permissao, unidades, produtos, estoque, pedido, pagamento aprovado, pagamento negado e erros de regra de negocio.

11. Execucao

bundle install

ruby bin/rails db:prepare

ruby bin/rails server

Testes automatizados:

ruby bin/rails test

12. Checklist de entrega

```
| Item do roteiro | Evidencia no projeto |
| --- | --- |
| Repositorio com codigo-fonte | raiz do projeto Rails |
| README de execucao | `README.md` |
| Variaveis de ambiente | `.env.example` |
| Banco, migrations e seed | `db/migrate`, `db/schema.rb`, `db/seeds.rb` |
| Swagger/OpenAPI | `/api/v1/openapi.json` |
| Colecao Postman | `docs/raizes_api.postman_collection.json` |
| DER | `docs/der.svg` e `docs/der.md` |
| Casos de uso | `docs/diagrama_casos_uso.svg` |
| Arquitetura | `docs/diagrama_arquitetura.svg` |
| Classes/fluxo critico | `docs/diagrama_classes.svg` e `docs/diagrama_fluxo_pedido.svg` |
| Plano de testes | `docs/test_plan.md` |
| Documento PDF | `docs/documentacao_projeto.pdf` |
```

13. Conclusao

A API entregue fecha o fluxo operacional principal da rede e deixa a base pronta para evoluir fidelidade, promocoões e integrações externas reais. A entrega inclui código, banco, documentação, contrato OpenAPI, diagramas e evidências de teste.